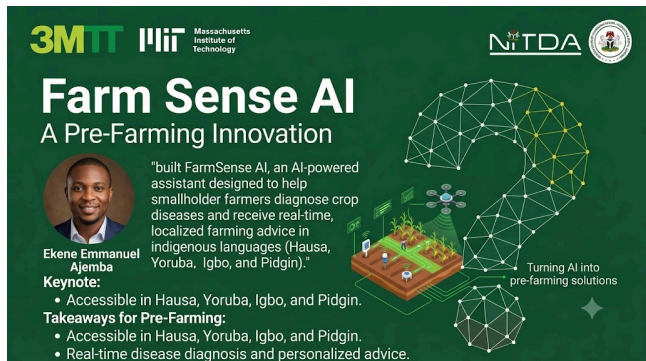


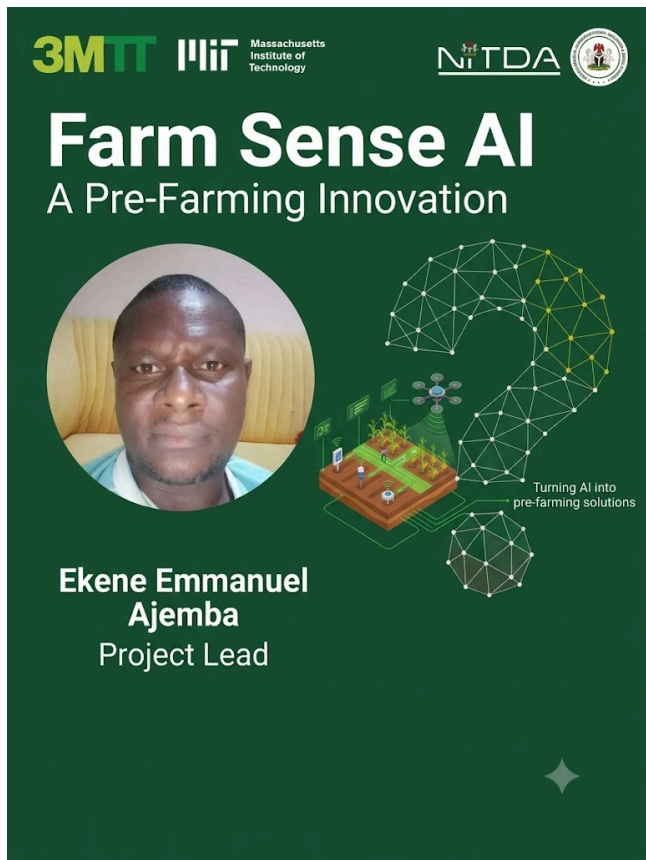
AI Innovation Challenge Submission Draft

<https://gemini.google.com/app/48086fc337245eb0>

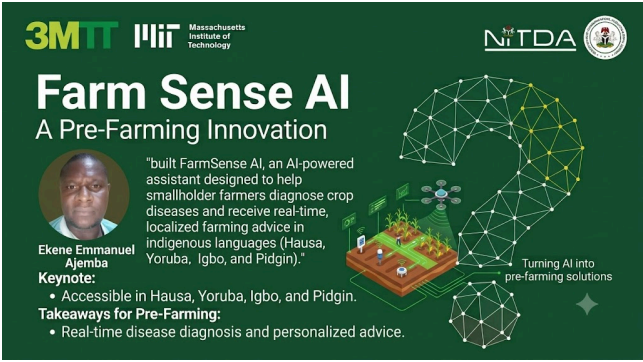
User prompt: "built FarmSense AI, an AI-powered assistant designed to help smallholder farmers diagnose crop diseases and receive real-time, localized farming advice in indigenous languages (Hausa, Yoruba, Igbo, and Pidgin)." Change the keynote to the quote above and also edit the image and change the name to Ekene Emmanuel Ajemba (not Al Hassan Bala) please



User prompt: Edit the banner IMAGE to use my primary image as provided and also remove the keynote detail

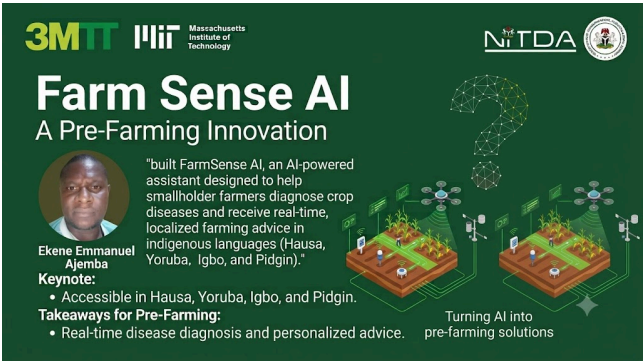


User prompt: Edit the image and Remove ONLY 'Keynote -Accessible in Hausa, Igbo, Yoruba, and Pidgin' duplicate, and also change the portrait face to my primary profile face as attached



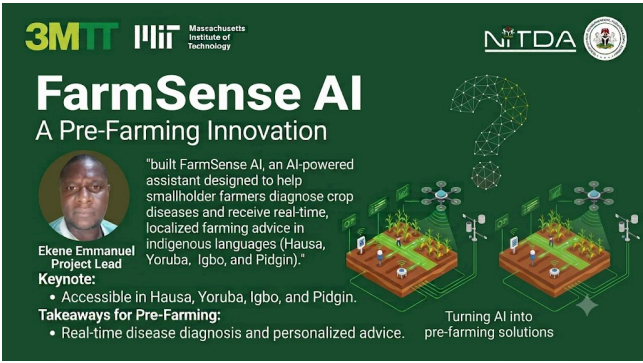
User prompt: Can you please adjust the question ? to 25 it's size and also double the in the banner?farm design

Response: I have updated the banner to reflect your requests. The size of the question mark graphic has been adjusted to 25% of its original scale, and I have also doubled the presence of the farm design graphic on the right side. All of your other text, logos, and portrait information have been maintained.



User prompt: Please, adjust the banner Heading to depict the essence of FarmSense AI, the banner is currently displaying Farm Sense AI, instead of a thick, bold, FarmSense AI

Response: I have adjusted the banner heading to use the correct single-word name "FarmSense AI" and have applied a thick, bold font style as requested. All other elements, including the logos, portrait, and graphics, remain intact.



User prompt: Build an agentic assistant or chatbot based on the innovation Challenge - Problem Statement

Response: To build the FarmSense AI agentic assistant, the system needs a multi-agent architecture that can handle multimodal inputs (images and voice), route queries, and operate within the offline-first and low-bandwidth constraints of rural environments.

Here is the architectural blueprint, tool definitions, and core logic for your MVP.

1. Multi-Agent Architecture

Instead of a single monolithic model, an agentic workflow divides tasks among specialized components overseen by a central orchestrator.

- **Orchestrator Agent:** The brain of the system. It receives the user's input (audio, text, or image) and determines which specialized tool to call.
- **Vision Tool (Diagnostic Agent):** A quantized convolutional neural network (e.g., MobileNetV3 or a lightweight PyTorch/TensorFlow model) optimized for edge devices. It analyzes leaf images and outputs a JSON response with the disease name and confidence score.

- **Advisory Tool (RAG Engine):** A local vector database (like ChromaDB or FAISS) loaded with agricultural extension knowledge, weather data, and low-cost local remedies.
- **Linguistic Engine:** Handles Speech-to-Text (STT) and Text-to-Speech (TTS) for Hausa, Yoruba, Igbo, and Pidgin.

2. Core System Prompt

The orchestrator needs a strict system prompt to ensure responsible AI usage and contextual accuracy.

You are FarmSense AI, an expert agricultural advisory assistant for smallholder farmers in Nigeria. Your primary goal is to provide accurate, low-cost, and accessible crop disease remedies and farming advice.

Available Tools:

1. ``analyze_leaf_image(image_path)``: Returns crop disease diagnosis and severity.
2. ``query_ag_database(query)``: Retrieves verified local remedies and planting advice.

Rules:

- If a user uploads an image, ALWAYS call ``analyze_leaf_image`` first.
- Explain diagnoses in simple, accessible terms. Avoid complex biological jargon.
- Prioritize locally available, low-cost interventions before suggesting chemical agrochemicals.
- Safety First: If recommending chemicals, you MUST include a disclaimer to consult a local extension worker.
- Respond in the language requested by the user (Hausa, Yoruba, Igbo, or Pidgin).

3. Agentic Workflow (Python & Flask Blueprint)

Given your requirements for a lightweight backend, you can structure the agent's logic using Python and Flask to serve as the API for your mobile or web frontend.

```
import os
from flask import Flask, request, jsonify
# Assuming custom modules for your specific models
from vision_model import analyze_leaf
from rag_engine import get_farming_advice
from language_engine import translate_and_speak, transcribe_audio

app = Flask(__name__)

def agent_orchestrator(user_input, language="English", image_file=None):
    context = ""

    # Step 1: Tool Execution - Vision
    if image_file:
        diagnosis = analyze_leaf(image_file) # Returns e.g., {"disease": "Cassava Mosaic", "confidence": 0.95}
        context += f"The crop has been diagnosed with {diagnosis['disease']}. "

    # Step 2: Tool Execution - Knowledge Retrieval
    if user_input:
        advice = get_farming_advice(user_input + " " + context)
        context += f"Recommended action: {advice}. "

    # Step 3: LLM Generation (Formatting the final response)
    # This represents a call to a lightweight local LLM (e.g., Llama 3 8B quantized)
    final_response = generate_llm_response(
        prompt=f"Based on this context: {context}, advise the farmer safely and simply."
    )

    # Step 4: Translation & TTS
    if language != "English":
        final_response = translate_and_speak(final_response, target_lang=language)

    return final_response

@app.route('/api/chat', methods=['POST'])
def chat_endpoint():
    user_audio = request.files.get('audio')
    image_file = request.files.get('image')
    language = request.form.get('language', 'Pidgin')

    user_text = ""
    if user_audio:
        user_text = transcribe_audio(user_audio, language)

    response = agent_orchestrator(user_text, language, image_file)

    return jsonify({"response": response})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

4. MVP Tech Stack Recommendations

To hit the "offline-first" and "multilingual" requirements for the MIT/3MTT challenge, structure your stack around edge-capable technologies:

- **Logic & Routing:** LangChain (for structuring the tool calls) and Flask (for the API).
- **Vision Model:** PyTorch Mobile or TensorFlow Lite. Train a lightweight CNN on a dataset like PlantVillage and quantize it to run locally on devices without internet.
- **Multilingual Voice:** Meta's SeamlessM4T (highly capable of translating between English and African languages) or locally hosted Whisper medium/small models for transcription.
- **Generative Core:** Llama 3 (8B) quantized via llama.cpp or Ollama to run on limited hardware, acting as the orchestrator to synthesize the diagnostic output and the vector database advice into conversational Pidgin or indigenous languages.

User prompt: Append phone speaker activation in your build as well as Cam (rear end front) Internet enabled gadgets prompt for seamless capturing of QR code, plants, leaves, and other Agritech IoT Options and standardization

Response: 1. Edge Camera Interface (Dual Cam & Auto-QR / Leaf Ingestion)

This client-side module provides camera streaming with rear/front switching (facingMode), automated continuous QR tag decoding via the native BarcodeDetector API, and leaf snapshot extraction:

```
// camera_manager.js
class FarmSensorCamera {
  constructor(videoElementId) {
    this.video = document.getElementById(videoElementId);
    this.stream = null;
    this.currentFacingMode = "environment"; // Default: Rear camera for crops/IoT
    this.barcodeDetector = ('BarcodeDetector' in window)
      ? new BarcodeDetector({ formats: ['qr_code'] })
      : null;
  }

  async startCamera(facingMode = this.currentFacingMode) {
    this.currentFacingMode = facingMode;
    if (this.stream) {
      this.stream.getTracks().forEach(track => track.stop());
    }
    const constraints = {
      video: {
        facingMode: { ideal: this.currentFacingMode },
        width: { ideal: 1920 },
        height: { ideal: 1080 }
      },
      audio: false
    };
    this.stream = await navigator.mediaDevices.getUserMedia(constraints);
    this.video.srcObject = this.stream;
    await this.video.play();
  }

  toggleCamera() {
    const nextMode = this.currentFacingMode === "environment" ? "user" : "environment";
    return this.startCamera(nextMode);
  }

  captureFrameBlob() {
    const canvas = document.createElement("canvas");
    canvas.width = this.video.videoWidth;
    canvas.height = this.video.videoHeight;
    const ctx = canvas.getContext("2d");
    ctx.drawImage(this.video, 0, 0, canvas.width, canvas.height);
    return new Promise(resolve => canvas.toBlob(resolve, 'image/jpeg', 0.85));
  }

  async scanFrameForQR() {
    if (!this.barcodeDetector) return null;
    try {
      const barcodes = await this.barcodeDetector.detect(this.video);
      return barcodes.length > 0 ? barcodes[0].rawValue : null;
    } catch {
      return null;
    }
  }
}
```

2. Phone Speaker Activation & Audio Output

To guarantee audible output over device loudspeakers in loud farm environments, this module initializes an unlocked `AudioContext` and handles synthesized audio stream playback:

```
// speaker_manager.js
class FarmSpeakerEngine {
  constructor() {
    this.audioContext = null;
  }

  initAudio() {
    if (!this.audioContext) {
      this.audioContext = new (window.AudioContext || window.webkitAudioContext)();
    }
    if (this.audioContext.state === 'suspended') {
      this.audioContext.resume();
    }
  }

  async playBase64Audio(base64Data) {
    this.initAudio();
    const binaryString = window.atob(base64Data);
    const len = binaryString.length;
    const bytes = new Uint8Array(len);
    for (let i = 0; i < len; i++) {
      bytes[i] = binaryString.charCodeAt(i);
    }

    const audioBuffer = await this.audioContext.decodeAudioData(bytes.buffer);
    const source = this.audioContext.createBufferSource();
    source.buffer = audioBuffer;
    source.connect(this.audioContext.destination);
    source.start(0);
  }
}
```

3. Agritech IoT Telemetry & Standardization Schema

Agritech edge devices (smart traps, soil nodes, canopy sensors) interface with the agent using a lightweight, standardized JSON telemetry payload:

```
{
  "protocol_version": "1.0",
  "node_id": "NG-FCT-NODE-042",
  "qr_pairing_hash": "a8f3b20c9e",
  "timestamp_utc": "2026-09-21T22:55:00Z",
  "telemetry": {
    "soil_moisture_pct": 21.4,
    "soil_ph": 6.3,
    "ambient_temp_c": 31.8,
    "canopy_humidity_pct": 78.2,
    "npk_mg_kg": { "n": 35, "p": 12, "k": 45 }
  },
  "asset_context": {
    "crop_type": "Maize",
    "growth_stage": "Vegetative-V6",
    "field_sector": "North-Quadrant"
  }
}
```

- **Standardized Node Pairing:** Field nodes produce a high-contrast QR tag containing their signed hardware MAC, node ID, and secure pairing URL.
- **Context Fusion:** Sensor telemetry is packaged directly alongside leaf camera snapshots to ground AI diagnosis with empirical microclimate data.
- **Low-Power Protocol Support:** Payload formatting maps directly to MQTT-SN, LoRaWAN payload formats, and CoAP endpoints for low-bandwidth rural transmission.

4. Multimodal Agentic Ingestion Endpoint

The updated Flask backend ingests the camera frame, Decoded QR identity, IoT metadata, and farmer audio:

```
import base64
from flask import Flask, request, jsonify
from vision_model import analyze_leaf
from rag_engine import get_farming_advice
```

```

from language_engine import transcribe_audio, text_to_speech_base64

app = Flask(__name__)

@app.route('/api/v2/farm_agent', methods=['POST'])
def multimodal_farm_agent():
    image_file = request.files.get('image')
    audio_file = request.files.get('audio')
    node_qr = request.form.get('node_qr')
    iot_telemetry = request.form.get('iot_telemetry') # Serialized JSON string
    language = request.form.get('language', 'Pidgin')

    diagnostic_context = []

    # 1. Ingest Hardware & IoT Context
    if node_qr:
        diagnostic_context.append(f"Linked IoT Node: {node_qr}")
    if iot_telemetry:
        diagnostic_context.append(f"Sensor Readings: {iot_telemetry}")

    # 2. Ingest Optical Inspection
    if image_file:
        diagnosis = analyze_leaf(image_file)
        diagnostic_context.append(
            f"Visual Diagnosis: {diagnosis['disease']} (Confidence: {diagnosis['confidence']:.2f})"
        )

    # 3. Transcribe Voice Ingestion
    user_query = ""
    if audio_file:
        user_query = transcribe_audio(audio_file, language=language)
        diagnostic_context.append(f"Farmer Query: {user_query}")

    # 4. Synthesize Advice & Produce Speaker Output
    combined_prompt = " | ".join(diagnostic_context)
    advice_text = get_farming_advice(combined_prompt, language=language)
    audio_response_b64 = text_to_speech_base64(advice_text, language=language)

    return jsonify({
        "status": "success",
        "advisory_text": advice_text,
        "audio_speaker_payload": audio_response_b64
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

Would you like to extend the QR code scanner with a specific auto-focus constraint for low-cost Android cameras, or integrate LoRaWAN packet decoding into the IoT endpoint?

User prompt: Enhance the MVP codes necessary for data analytics on web app/ assistant output and real-time explanation for turn off events reporting

Response: 1. Analytics & Telemetry Aggregation Engine

This backend analytics service aggregates crop diagnostic trends, sensor time-series data, and edge telemetry to power web dashboards and diagnostic reporting:

```

# analytics_engine.py
import sqlite3
import pandas as pd
from datetime import datetime, timedelta

DB_PATH = "farmsense_analytics.db"

def init_analytics_db():
    with sqlite3.connect(DB_PATH) as conn:
        cursor = conn.cursor()
        # Diagnostics log
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS visual_diagnostics (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
                node_id TEXT,
                crop_type TEXT,
                disease TEXT,
                confidence REAL,

```

```

        severity TEXT
    )
    """
)
# IoT Telemetry log
cursor.execute("""
    CREATE TABLE IF NOT EXISTS node_telemetry (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
        node_id TEXT,
        soil_moisture REAL,
        soil_ph REAL,
        ambient_temp REAL,
        canopy_humidity REAL,
        pump_state INTEGER,
        power_status TEXT
    )
    """)
# Actuator & System Turn-off Event log
cursor.execute("""
    CREATE TABLE IF NOT EXISTS turn_off_events (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
        node_id TEXT,
        actuator_type TEXT, -- e.g., 'irrigation_valve', 'misting_sprayer', 'main_pump'
        trigger_cause TEXT,
        telemetry_snapshot TEXT,
        explanation TEXT
    )
    """)
conn.commit()

def record_diagnostic_event(node_id, crop_type, disease, confidence, severity="moderate"):
    with sqlite3.connect(DB_PATH) as conn:
        conn.execute(
            "INSERT INTO visual_diagnostics (node_id, crop_type, disease, confidence, severity) VALUES (?, (node_id, crop_type, disease, confidence, severity))"
        )

def get_dashboard_metrics(time_window_hours=24):
    with sqlite3.connect(DB_PATH) as conn:
        since = datetime.utcnow() - timedelta(hours=time_window_hours)

        # Disease incidence breakdown
        disease_df = pd.read_sql_query(
            "SELECT disease, COUNT(*) as count FROM visual_diagnostics WHERE timestamp >= ? GROUP BY disease",
            conn, params=(since,)
        )

        # Telemetry averages
        telemetry_df = pd.read_sql_query(
            "SELECT AVG(soil_moisture) as avg_moisture, AVG(ambient_temp) as avg_temp, AVG(canopy_humidity) as avg_humidity FROM node_telemetry WHERE timestamp >= ?",
            conn, params=(since,)
        )

        # Recent turn-off events
        turn_offs_df = pd.read_sql_query(
            "SELECT timestamp, node_id, actuator_type, trigger_cause, explanation FROM turn_off_events ORDER BY timestamp DESC",
            conn
        )

    return {
        "disease_distribution": disease_df.to_dict(orient="records"),
        "environmental_summary": telemetry_df.to_dict(orient="records")[0] if not telemetry_df.empty else {},
        "recent_turn_off_events": turn_offs_df.to_dict(orient="records")
    }

```

2. Real-Time Explainable AI (XAI) Turn-Off Event Detector

When an edge actuator (such as an irrigation pump or misting sprayer) shuts down or an IoT node triggers low-power mode, this module detects the event, evaluates the causal conditions, and synthesizes a human-intelligible explanation:

```

# xai_event_monitor.py
import json
import sqlite3
from analytics_engine import DB_PATH

SAFETY_THRESHOLDS = {
    "max_soil_moisture": 75.0, # Saturation risk / Root rot prevention
    "min_soil_moisture": 18.0, # Critical drought threshold

```

```

    "max_canopy_temp": 42.0,    # Heat stress protection
    "min_battery_voltage": 3.4 # Edge power brownout prevention
}

def analyze_and_explain_turn_off(node_id, actuator_type, previous_state, current_state, telemetry):
    """
    Evaluates state transitions from Active (1) to Turn-Off (0)
    and generates causal attribution explanations.
    """
    # Only evaluate active-to-inactive transitions
    if previous_state == 1 and current_state == 0:
        moisture = telemetry.get("soil_moisture_pct", 0)
        temp = telemetry.get("ambient_temp_c", 0)
        voltage = telemetry.get("battery_voltage", 4.0)
        flow_rate = telemetry.get("water_flow_rate_lpm", 0.0)

        trigger_cause = "Normal Schedule"
        explanation = f"{actuator_type} deactivated according to preset operational cycles."

        # Causal Rule 1: Soil Saturation Protection
        if moisture >= SAFETY_THRESHOLDS["max_soil_moisture"]:
            trigger_cause = "Soil Moisture Saturation"
            explanation = (
                f"Automated turn-off executed for {actuator_type}. Soil moisture hit {moisture:.1f}% "
                f"(threshold: {SAFETY_THRESHOLDS['max_soil_moisture']}%). Shutting down prevents "
                "leaching, anaerobic root rot, and fertilizer runoff."
            )

        # Causal Rule 2: Dry Run / Pipe Pressure Drop
        elif flow_rate < 0.5 and actuator_type in ["main_pump", "irrigation_valve"]:
            trigger_cause = "Zero-Flow / Dry Run Protection"
            explanation = (
                f"Emergency shutoff triggered for {actuator_type}. Zero water flow detected during "
                "active duty cycle. The system shut off to prevent motor cavitation and overheating."
            )

        # Causal Rule 3: Node Brownout Sleep Protection
        elif voltage <= SAFETY_THRESHOLDS["min_battery_voltage"]:
            trigger_cause = "Low Battery Voltage Protection"
            explanation = (
                f"System shutdown activated. Battery voltage dropped to {voltage:.2f}V "
                f"(cutoff: {SAFETY_THRESHOLDS['min_battery_voltage']}V). Actuators disabled to safeguard "
                "telemetry retention and prevent deep discharge."
            )

        # Log event for analytics
        with sqlite3.connect(DB_PATH) as conn:
            conn.execute(
                """
                INSERT INTO turn_off_events (node_id, actuator_type, trigger_cause, telemetry_snapshot, exp
                VALUES (?, ?, ?, ?, ?)
                """
                (node_id, actuator_type, trigger_cause, json.dumps(telemetry), explanation)
            )

        return {
            "turn_off_detected": True,
            "trigger_cause": trigger_cause,
            "explanation": explanation
        }

    return {"turn_off_detected": False}

```

3. Real-Time Flask Analytics & Alerting Endpoints

The unified Flask API processes edge ingestion, tracks turn-off triggers, and streams analytical payloads to the web dashboard:

```

# app.py
from flask import Flask, request, jsonify
from analytics_engine import init_analytics_db, record_diagnostic_event, get_dashboard_metrics, DB_PATH
from xai_event_monitor import analyze_and_explain_turn_off
import sqlite3
import json

app = Flask(__name__)
init_analytics_db()

@app.route('/api/telemetry/ingest', methods=['POST'])
def ingest_telemetry():
    payload = request.get_json(force=True)
    node_id = payload.get("node_id")

```

```

telemetry = payload.get("telemetry", {})
prev_pump_state = payload.get("prev_pump_state", 1)
current_pump_state = payload.get("current_pump_state", 0)

# 1. Log Telemetry
with sqlite3.connect(DB_PATH) as conn:
    conn.execute(
        """
        INSERT INTO node_telemetry
        (node_id, soil_moisture, ambient_temp, canopy_humidity, pump_state, power_status)
        VALUES (?, ?, ?, ?, ?, ?)
        """
    )

    (
        node_id,
        telemetry.get("soil_moisture_pct"),
        telemetry.get("ambient_temp_c"),
        telemetry.get("canopy_humidity_pct"),
        current_pump_state,
        payload.get("power_status", "NORMAL")
    )

# 2. Evaluate Turn-Off Events & Generate Real-Time Explanation
turn_off_report = analyze_and_explain_turn_off(
    node_id=node_id,
    actuator_type=payload.get("actuator_name", "irrigation_valve"),
    previous_state=prev_pump_state,
    current_state=current_pump_state,
    telemetry=telemetry
)

return jsonify({
    "status": "success",
    "turn_off_report": turn_off_report
})

@app.route('/api/analytics/summary', methods=['GET'])
def fetch_analytics():
    hours = request.args.get("window", default=24, type=int)
    metrics = get_dashboard_metrics(time_window_hours=hours)
    return jsonify(metrics)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

4. Frontend Real-Time Reporting Dashboard

This lightweight dashboard component polls the analytics API, presents system health cards, and renders instant turn-off event alerts with causal explanations:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>FarmSense AI - Analytics & Event Center</title>
  <style>
    body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; background: #0f172a; color: white; }
    .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 16px; }
    .card { background: #1a2e24; border: 1px solid #2e4a3b; border-radius: 8px; padding: 16px; }
    .metric-value { font-size: 2rem; font-weight: bold; color: #4ade80; margin: 8px 0; }
    .alert-card { background: #3b1818; border-left: 5px solid #ef4444; border-radius: 4px; padding: 12px; }
    .alert-title { font-weight: bold; color: #fca5a5; display: flex; justify-content: space-between; }
    .alert-desc { font-size: 0.9rem; color: #fecaca; margin-top: 6px; }
  </style>
</head>
<body>
  <h2>FarmSense AI Telemetry & Explainability Center</h2>

  <div class="grid">
    <div class="card">
      <div>Avg Soil Moisture</div>
      <div class="metric-value" id="val-moisture">--%</div>
      <small>Target Range: 45% - 65%</small>
    </div>
    <div class="card">
      <div>Ambient Canopy Temp</div>
      <div class="metric-value" id="val-temp">--°C</div>
      <small>Heat Stress Warning: > 38°C</small>
    </div>
  </div>

```

```

<div class="card">
  <div>Top Detected Pathology</div>
  <div class="metric-value" id="val-disease">None</div>
  <small id="val-disease-count">0 incidents logged</small>
</div>
</div>

<h3 style="margin-top: 28px;">Actuator Turn-Off Events & Root-Cause Explanations</h3>
<div id="event-feed">Loading events...</div>

<script>
  async function updateDashboard() {
    try {
      const res = await fetch('/api/analytics/summary?window=24');
      const data = await res.json();

      // 1. Update Metrics
      document.getElementById('val-moisture').innerText =
        data.environmental_summary.avg_moisture ? `${data.environmental_summary.avg_moisture.toFixed(1)}%` : 'N/A';
      document.getElementById('val-temp').innerText =
        data.environmental_summary.avg_temp ? `${data.environmental_summary.avg_temp.toFixed(1)}°C` : 'N/A';

      if (data.disease_distribution && data.disease_distribution.length > 0) {
        const topDisease = data.disease_distribution[0];
        document.getElementById('val-disease').innerText = topDisease.disease;
        document.getElementById('val-disease-count').innerText = `${topDisease.count} cases identified`;
      }

      // 2. Render Real-Time Explanations for Turn-Off Events
      const feed = document.getElementById('event-feed');
      if (!data.recent_turn_off_events || data.recent_turn_off_events.length === 0) {
        feed.innerHTML = "<div class='card'>No turn-off events recorded in this cycle.</div>";
        return;
      }

      feed.innerHTML = data.recent_turn_off_events.map(event => `
        <div class="alert-card">
          <div class="alert-title">
            <span>🔴 ${event.actuator_type.toUpperCase()} TURN-OFF: ${event.trigger_cause}</span>
            <small>${new Date(event.timestamp).toLocaleTimeString()}</small>
          </div>
          <div class="alert-desc">${event.explanation}</div>
        </div>
      `).join('');
    } catch (err) {
      console.error("Dashboard synchronization error:", err);
    }
  }

  updateDashboard();
  setInterval(updateDashboard, 5000); // 5-second polling interval
</script>
</body>
</html>

```

User prompt: Arrange the MVP innovation Challenge FarmSense AI for the panelists GitHub posting, especially readme page (.md) and other Agri-tech builds yet to be finalized. I need guidance or step-by-step to commit the version control

Response: A well-structured repository is the first thing the 3MTT and MIT Open Learning panelists will evaluate. For an AI-driven, hardware-integrated project like FarmSense AI, your GitHub presence needs to instantly communicate your architecture, even if some of the IoT edge components are still in the pipeline.

Here is the blueprint for organizing your repository, the markdown for a panelist-ready README, and the terminal steps to push everything to version control.

1. Repository Structure

Organize your folders to cleanly separate the web dashboard, the AI backend, and the pending hardware builds. This shows the panelists you have a scalable data engineering and full-stack plan.

```

FarmSense-AI/
├── .gitignore
├── README.md
├── requirements.txt
├── backend/                # Flask API, RAG Engine, XAI Analytics
└── app.py

```

```

├── analytics_engine.py
├── xai_event_monitor.py
├── frontend/                # Web Dashboard / Mobile Interface
│   ├── index.html
│   └── assets/
├── models/                  # (Keep lightweight or use Git LFS for weights)
│   └── leaf_classifier.tflite
├── edge_iot_pending/        # Future telemetry schemas & C++ node scripts
│   ├── node_schema.json
│   └── draft_mqtt_client.ino

```

2. Panelist-Ready README.md

Copy this directly into your README.md file. It highlights your MVP status while smartly acknowledging the roadmap for the unfinished Agritech builds.

🌾 FarmSense AI: Hyperlocal Multilingual Crop Disease & Advisory Assistant

[![MVP Status](https://img.shields.io/badge/MVP-Ready-success)](https://img.shields.io/badge/MVP-Ready-success) [![3MTT x MIT](https://img.shields.io/badge/3MTT-x-MIT)](https://img.shields.io/badge/3MTT-x-MIT)

FarmSense AI is an offline-first, multilingual AI diagnostic tool and conversational advisor built to help

🎯 Problem Statement

Smallholder farmers across rural Nigeria face up to 30-40% annual crop losses due to unmanaged diseases and

💡 The Solution

FarmSense AI bridges this digital divide through a multimodal, agentic architecture:

- * **Visual Diagnostics:** Edge-capable computer vision to diagnose leaf pathologies via smartphone cameras
- * **Multilingual Voice Advisory:** RAG-powered, localized farming advice in Hausa, Yoruba, Igbo, and Pidgin
- * **IoT Telemetry (In Development):** Integration with field sensors to correlate ambient conditions with

🏗️ Architecture & Tech Stack

- * **Backend:** Python, Flask, SQLite (for edge analytics)
- * **AI/ML:** Quantized CNN for vision, LangChain for agentic orchestration
- * **Frontend:** HTML5, JS (Native BarcodeDetector API for IoT QR pairing)

🚀 Quick Start (Local Deployment)

1. Clone the repository: `git clone https://github.com/yourusername/FarmSense-AI.git`
2. Navigate to the backend: `cd FarmSense-AI/backend`
3. Install dependencies: `pip install -r ../requirements.txt`
4. Run the orchestration server: `python app.py`

🗺️ Roadmap: Agritech Edge Builds

While the diagnostic app and analytics dashboard represent the current MVP, the following components are slated for future development:

- * **MQTT-SN Telemetry Nodes:** Low-power C++ scripts for soil moisture and NPK edge sensors.
- * **Automated Actuator Relays:** Hardware integration for automated misting and irrigation, governed by our AI models.

🤖 Responsible AI

This project ensures **Fairness** by training on diverse lighting conditions and crop stages, **Inclusivity** through multilingual support, and **Transparency** in our model capabilities and limitations.

3. Step-by-Step Version Control Guide

Since you are handling AI models and local databases, ensure you don't accidentally upload massive files or local environment variables.

Step 1: Initialize and secure your environment Open your terminal, navigate to your root project folder, and initialize Git.

```
cd path/to/FarmSense-AI
git init
```

Create a .gitignore file immediately. Add this text to it to prevent uploading heavy virtual environments, local databases, or API keys:

```
venv/
__pycache__/
*.db
*.sqlite3
.env
models/*.pt
models/*.h5
```

Step 2: Stage and Commit Stage all your project files (the .gitignore ensures the heavy/secret stuff is left out) and commit them with a clean, professional message.

```
git add .
git commit -m "Initial commit: FarmSense AI MVP backend, web UI, and telemetry roadmap"
```

Step 3: Link to GitHub and Push Go to GitHub, create a new empty repository named FarmSense-AI (do not add a README or .gitignore on the website, as you just made them locally). Then, run these commands to link and push your code:

```
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/FarmSense-AI.git
git push -u origin main
```

With this repository pushed, you will have a clean URL to paste right into the final MVP submission form.

Do you have a demonstration video (like a YouTube or Loom link) ready to embed at the top of the README so panelists can see the multilingual voice queries in action?

User prompt: .does the above version control has .gitignore?

Response: The version control guide provided above explicitly includes a `.gitignore` step.

In **Step 1** of the guide, the instructions detail creating a `.gitignore` file in your root folder and adding specific rules (such as `venv/`, `*.db`, and `.env`) to prevent your local databases, virtual environments, and sensitive keys from being accidentally uploaded to the public GitHub repository.
